Лабораторная работа №6 **SharedFlow**

Мы познакомились с Flow – потоками данных Kotlin и узнали, как их можно использовать для возврата нескольких последовательных значений из асинхронного кода на основе корутин. Далее мы рассмотрим более подробную реализацию потока, на этот раз с использованием SharedFlow. Также будет показано, как обеспечить корректную работу коллекции потоков при переключении приложения между фоновым и основным режимами.

1 О проекте

Созданное приложение будет состоять из пользовательского интерфейса, содержащего компонент List. Мы активируем общий поток во ViewModel сразу после создания модели представления и будем генерировать целочисленное значение каждые две секунды. Основное действие будет собирать значения из потока и отображать их в List. Затем мы изменим проект, чтобы приостанавливать процесс сбора данных, когда приложение находится в фоновом режиме.

2 Создание проекта SharedFlowDemo

Запустите Android Studio и создайте новый *Empty Activity* проект под названием SharedFlowDemo, указав *com.example.sharedflowdemo* в качестве имени пакета и выбрав минимальный уровень API 27: Android 8.1 (Oreo).

В файле *MainActivity.kt* удалите функцию Greeting и добавьте новую composable функцию с именем ScreenSetup, которая, в свою очередь, вызывает функцию с именем MainScreen:

```
@Composable
fun ScreenSetup(modifier: Modifier = Modifier) {
   MainScreen(modifier)
}
@Composable
fun MainScreen(modifier: Modifier = Modifier) {
}
```

Отредактируйте функцию метода *OnCreate()*, чтобы она вызывала ScreenSetup вместо Greeting, и удалите вызов Greeting из GreetingPreview.

Далее измените файл *libs.versions.toml*, чтобы добавить библиотеку Compose ViewModel в каталог версий:

```
androidx-lifecycle-viewmodel-compose = {
      module = "androidx.lifecycle:lifecycle-viewmodel-compose",
      version.ref = "lifecycleRuntimeKtx" }
```

Отредактируйте файл *build.gradle.kts (Module :app)* и добавьте библиотеку в раздел зависимостей следующим образом: **implementation(libs.androidx.lifecycle.viewmodel.compose)**

Когда появится запрос, нажмите кнопку «Sync Now» в верхней части панели редактора, чтобы зафиксировать изменения.

3 Добавление ViewModel в проект

В этом проекте поток снова будет реализован в классе ViewModel. Добавьте эту модель в проект, найдя и щелкнув правой кнопкой мыши запись *app -> kotlin+java -> com.example.sharedflowdemo* в окне инструментов проекта и выбрав пункт меню *New -> Kotlin Class/File*. В появившемся диалоговом окне назовите класс *DemoViewModel*, а затем нажмите клавишу Enter. После создания файла измените его содержимое следующим образом:

```
import androidx.lifecycle.ViewModel

class DemoViewModel : ViewModel() {
}
```

Вернитесь в файл *MainActivity.kt* и внесите изменения для доступа к экземпляру ViewModel:

```
@Composable
fun ScreenSetup(
   modifier: Modifier = Modifier,
   viewModel: DemoViewModel = viewModel()
) {
   MainScreen(modifier)
}
```

4 Объявление SharedFlow

Следующий шаг – добавить в модель представления код для создания и запуска экземпляра SharedFlow. Начните с редактирования файла *DemoViewModel.kt*, чтобы он выглядел следующим образом:

```
class DemoViewModel : ViewModel() {

   private val _sharedFlow = MutableSharedFlow<Int>()
   val sharedFlow = _sharedFlow.asSharedFlow()

   init {
      sharedFlowInit()
   }

   fun sharedFlowInit() {
   }
}
```

При создании экземпляра ViewModel инициализатор вызовет функцию *sharedFlowInit()*. Эта функция предназначена для запуска новой корутины, содержащей цикл, в котором с помощью общего потока передаются новые значения.

После объявления потока мы можем добавить код в функцию *sharedFlowInit()* для запуска потока с использованием области видимости модели представления. Это гарантирует, что поток завершится при уничтожении модели представления:

```
private fun sharedFlowInit() {
   viewModelScope.launch {
      for (i in 1..1000) {
         delay(2000)
         _sharedFlow.emit(i)
      }
   }
}
```

5 Сбор данных о потоке

Перед первым тестированием приложения нам нужно добавить код для сбора данных из потока и отображения этих значений в composable элементе LazyColumn. По мере сбора данных из потока мы будем добавлять их в изменяемый экземпляр списка, который, в свою очередь, будет служить источником данных для содержимого LazyColumn. Нам также нужно передать ссылку на общий поток в composable элемент MainScreen. Отредактируйте файл *MainActivity.kt* и внесите следующие изменения:

```
@Composable
fun ScreenSetup(
   modifier: Modifier = Modifier,
   viewModel: DemoViewModel = viewModel()
) {
```

```
    MainScreen(modifier, viewModel.sharedFlow)
}

@Composable
fun MainScreen(
    modifier: Modifier = Modifier,
    sharedFlow: SharedFlow<Int>
) {

    val messages = remember { mutableStateListOf<Int>()}

    LazyColumn(modifier = modifier) {
        items(messages) {
            Text(
                "Collected Value = $it",
                style = MaterialTheme.typography.headlineLarge,
                modifier = Modifier.padding(5.dp)
            )
        }
    }
}

@Preview(showBackground = true)
@Composable
fun GreetingPreview() {
    SharedFlowDemoTheme {
        val viewModel: DemoViewModel = viewModel()
        MainScreen(sharedFlow = viewModel.sharedFlow)
    }
}
```

После внесения этих изменений мы можем собирать значения, передаваемые общим потоком, и отображать их. Поскольку сбор данных из потока будет осуществляться в корутине и вне области действия composable элемента MainScreen, код запуска необходимо поместить в вызов LaunchedEffect (эта тема рассматривалась ранее). Добавьте вызов LaunchedEffect в composable элемент MainScreen, чтобы собирать данные из потока:

```
@Composable
fun MainScreen(
    modifier: Modifier = Modifier,
    sharedFlow: SharedFlow<Int>
) {
    val messages = remember { mutableStateListOf<Int>()}
    val lifecycleOwner = LocalLifecycleOwner.current

    LaunchedEffect(key1 = Unit) {
        sharedFlow.collect {
            messages.add(it)
        }
    }
```

Этот код обращается к общему экземпляру потока в модели представления и начинает собирать значения из потока. Каждое собранное значение добавляется в изменяемый список

messages. Это приведет к рекомпиляции, и новое значение появится в конце списка LazyColumn.

6 Тестирование приложения SharedFlowDemo

Скомпилируйте и запустите приложение на устройстве или в эмуляторе и убедитесь, что значения отображаются в LazyColumn по мере их передачи из общего потока. Поверните устройство в горизонтальное положение, чтобы вызвать изменение конфигурации, и убедитесь, что подсчет продолжается без обнуления:

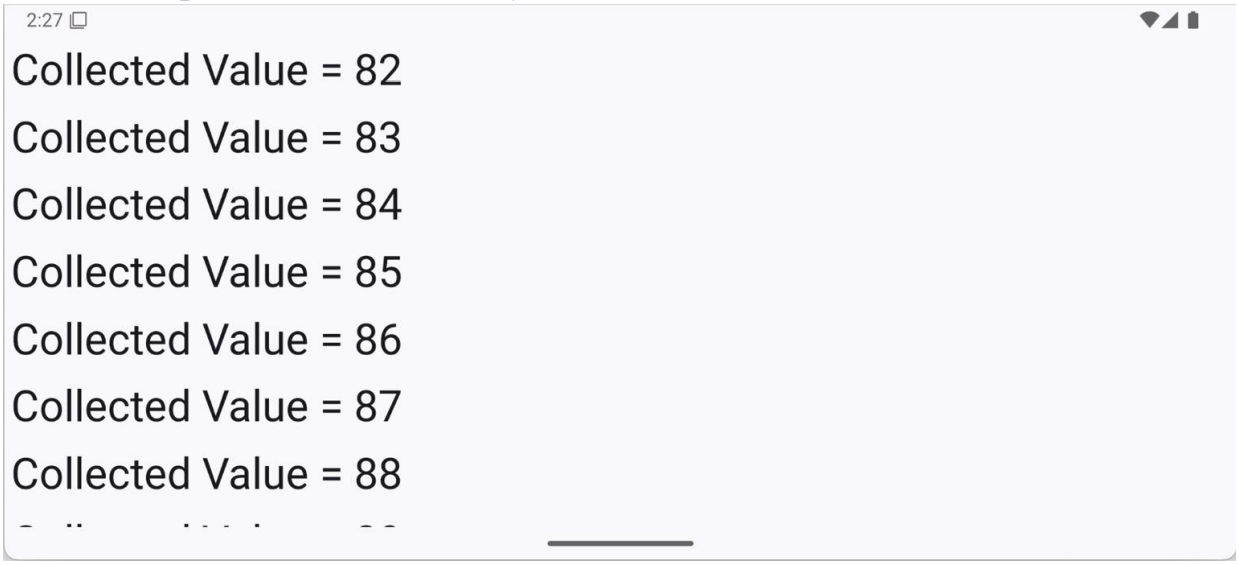


Рис. 1

Теперь, когда приложение работает, давайте посмотрим, что происходит, когда оно находится в фоновом режиме.

7 Обработка потоков в фоновом режиме

В нашем приложении есть общий поток, который передает значения в пользовательский интерфейс в виде LazyColumn. Благодаря выполнению сбора данных в области корутины пользовательский интерфейс остается отзывчивым во время сбора данных (вы можете убедиться в этом, прокручивая список значений вверх и вниз во время его обновления). В связи с этим возникает вопрос: что происходит, когда приложение переходит в фоновый режим? Чтобы выяснить это, мы можем добавить некоторые диагностические данные в код эмиттера и сборщика. Сначала отредактируйте файл *DemoViewModel.kt* и добавьте вызов *println()* в тело цикла *for*:

```
private fun sharedFlowInit() {
  viewModelScope.launch {
    for (i in 1..1000) {
      delay(2000)
      println("Emitting $i")
      _sharedFlow.emit(i)
    }
  }
}
```

Внесите аналогичное изменение в блок кода для сбора данных в файле *MainActivity.kt* следующим образом:

```
  LaunchedEffect(key1 = Unit) {
    sharedFlow.collect {
      println("Collecting $it")
      messages.add(it)
    }
  }
```

После внесения этих изменений откройте окно инструмента Logcat, введите *System.out* в строку поиска и запустите приложение. По мере обновления списка значений в окне инструмента Logcat должен появиться следующий вывод:

```
Emitting 1
Collecting 1
Emitting 2
Collecting 2
Emitting 3
Collecting 3
```

Теперь переведите приложение в фоновый режим и обратите внимание, что операции отправки и сбора данных продолжают выполняться, даже если приложение больше не видно пользователю. Продолжающаяся отправка данных является ожидаемым и правильным поведением для общего потока, реализованного в модели представления. Однако сбор данных и обновление пользовательского интерфейса, который в данный момент не виден пользователю, являются нерациональным использованием ресурсов. Мы можем решить эту проблему, выполнив сбор данных с помощью функции *repeatOnLifecycle*.

Функция *repeatOnLifecycle* – это приостанавливаемая suspend функция, которая запускает указанный блок кода каждый раз, когда текущий жизненный цикл достигает или превышает одно из следующих состояний:

•Lifecycle.State.INITIALIZED
•Lifecycle.State.CREATED
•Lifecycle.State.STARTED
•Lifecycle.State.RESUMED
•Lifecycle.State.DESTROYED

И наоборот, когда жизненный цикл опускается ниже целевого состояния, корутина отменяется.

В этом случае мы хотим, чтобы коллекция запускалась каждый раз, когда достигается *Lifecycle.State.STARTED,* и останавливалась, когда жизненный цикл приостанавливается. Чтобы реализовать это, измените код коллекции следующим образом:

```
import androidx.lifecycle.Lifecycle
import androidx.lifecycle.repeatOnLifecycle
.
.
LaunchedEffect(key1 = Unit) {
  lifecycleOwner.repeatOnLifecycle(Lifecycle.State.STARTED) {
    sharedFlow.collect {
      println("Collecting $it")
      messages.add(it)
    }
  }
}
```

Запустите приложение ещё раз, переведите его в фоновый режим и обратите внимание, что в выводе Logcat отображаются только диагностические сообщения об эмиссии, подтверждающие, что основное действие больше не собирает значения и не добавляет их в список RecyclerView. Когда приложение будет переведено в передний план, сбор данных возобновится с последнего отправленного значения, поскольку мы не настроили повторную отправку в общем потоке.

Резюме

В результате мы создали экземпляр SharedFlow в модели представления. Затем мы собрали потоковые значения в основной активити и использовали эти данные для обновления пользовательского интерфейса. Мы также подчеркнули важность предотвращения ненужных обновлений пользовательского интерфейса, управляемых потоком, когда приложение работает в фоновом режиме. Эту проблему можно легко решить с помощью функции *repeatOnLifecycle*. Мы можем использовать эту функцию для отмены и перезапуска асинхронных задач, таких как сбор потока, когда жизненный цикл приложения достигает целевого состояния.